

Planning and State

CMSC 848N

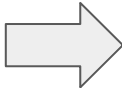
Fall 2026, Week 4, Lecture 1

Based on the EMNLP 2024 tutorial by Yu Su, Diyi Yang, Shunyu Yao, and Tao Yu

Acting in language



ALFRED



You are in the middle of a room. Looking quickly around you, you see a drawer 2, a shelf 5, a drawer 1, a shelf 4, a sidetable 1, a drawer 5, a shelf 6, a shelf 1, a shelf 9, a cabinet 2, a sofa 1, a cabinet 1, a shelf 3, a cabinet 3, a drawer 3, a shelf 11, a shelf 2, a shelf 10, a dresser 1, a shelf 12, a garbagecan 1, a armchair 1, a cabinet 4, a shelf 7, a shelf 8, a safe 1, and a drawer 4.

Your task is to: *put some vase in safe.*

> go to shelf 6

You arrive at loc 4. On the shelf 6, you see a vase 2.

> take vase 2 from shelf 6

You pick up the vase 2 from the shelf 6.

> go to safe 1

You arrive at loc 3. The safe 1 is closed.

> open safe 1

You open the safe 1. The safe 1 is open. In it, you see a keychain 3.

> put vase 2 in/on safe 1

You won!

ALFWORLD

ReAct: reasoning with observations

Question: Which magazine was started first, Arthur's Magazine or First for Women?

Thought 1: I need to search Arthur's Magazine and First for Women, and find which was started first.

Action 1: Search[Arthur's Magazine]

Observation 1: Arthur's Magazine (1844–1846) was an American literary periodical published in Philadelphia...

Thought 2: Arthur's Magazine was started in 1844. I need to search First for Women next.

Action 2: Search[First for Women]

Observation 2: First for Women is a woman's magazine published by Bauer Media Group... was started in 1989...

Thought 3: First for Women was started in 1989. 1844 (Arthur's Magazine) < 1989 (First for Women), so Arthur's Magazine was started first.

Action 3: Finish[Arthur's Magazine]

The same question with different prompts

Which magazine was started first, Arthur's Magazine or First for Women?

Reason only
(chain of thought)

Thought: Arthur's Magazine was started in 1844. First for Women was started in 1989.
Since $1844 < 1989$, Arthur's Magazine was started first.
Answer: Arthur's Magazine

Act only

Action 1: Search[Arthur's Magazine]
Observation 1: Arthur's Magazine (1844–1846) was an American literary periodical published in Philadelphia...
Action 2: Search[First for Women]
Observation 2: First for Women is a woman's magazine published by Bauer Media Group... was started in 1989...
Action 3: Finish[Arthur's Magazine]

Standard prompt

Answer: Arthur's Magazine

Planning: a working definition

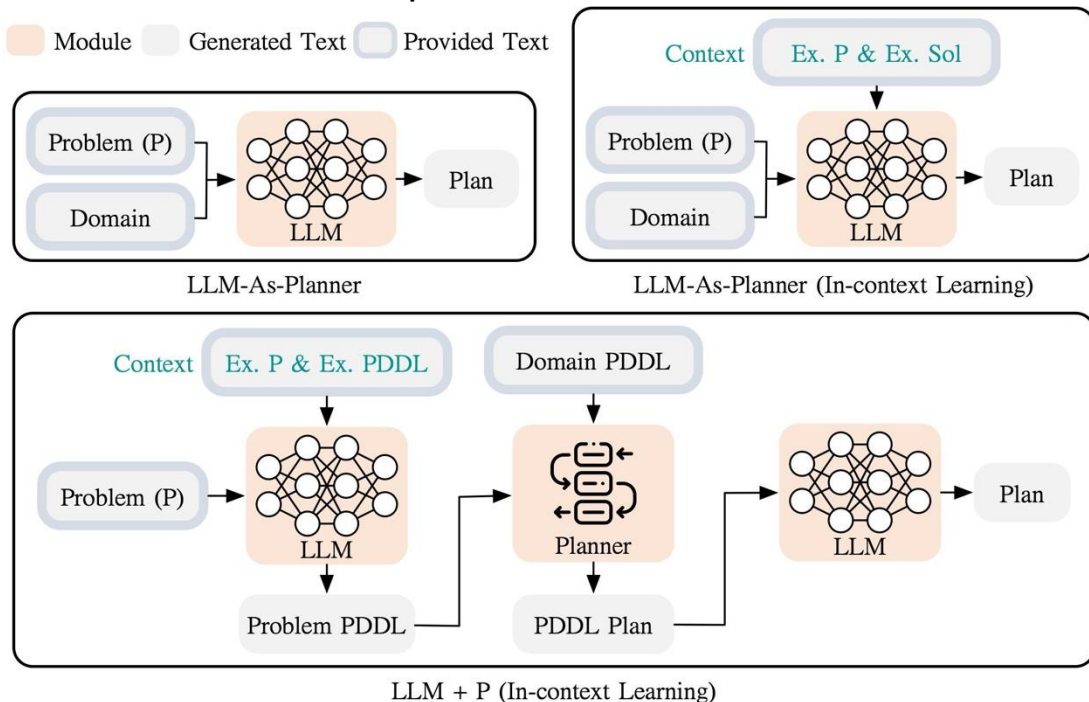
Given a goal G , decides on a sequence of actions $(a_0, a_1, \dots, a_n)$ that will lead to a state that passes the goal test $g(\cdot)$

What changes for language agents?

- Goals may arrive as natural-language instructions.
- Available actions may be numerous or open-ended.
- An automated test may not clearly establish success.

LLM+P

The LLM translates the problem into PDDL. A classical planner searches for a plan.



An Example PDDL Problem File Written by GPT-4 with Context

Context (shaded):

An example planning problem is: You have 5 blocks. b2 is on top of b5. b5 is on top of b1. b1 is on top of b4. b3 is on top of b2. b4 is on the table. b3 is clear. Your arm is empty. Your goal is to move the blocks. b4 should be on top of b3. The problem PDDL is:

```
(:objects b1 b2 b3 b4 b5)
(:init (arm-empty) (on b1 b4) (on
b2 b5) (on b3 b2)
(on-table b4) (on b5 b1)
(clear b3))
(:goal (and (on b4 b3))))
```

Prompt: Description of Problem (P1) + Context + Provide me with the problem PDDL file that describes the planning problem directly without further explanations.

GPT-4 (the generated problem PDDL):

```
(:objects b1 b2 b3 b4 b5 )
(:init (arm-empty) (on-table b1)
(on b2 b1) (on b3 b4)
(on b4 b2) (on b5 b3) (clear
b5)))
(:goal (and (on b1 b2) (on b3 b5)
(on b4 b1)))
```

Planner:

```
(unstack b5 b3) (putdown b5)
(unstack b3 b4) (stack b3 b5)
(unstack b4 b2) (putdown b4)
(unstack b2 b1) (putdown b2)
(pickup b1) (stack b1 b2) (pickup
b4) (stack b4 b1)
```

Planning a sequence of web actions

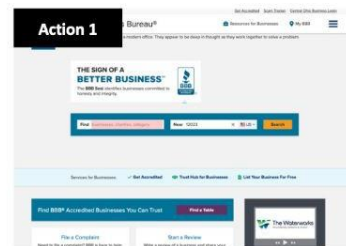
Task Description:

Show me the reviews for the auto repair business closest to 10002.

Action Sequence:

Target Element	Operation
1. [searchbox] Find	TYPE: auto repair
2. [button] Auto Repair	CLICK
3. [textbox] Near	TYPE: 10002
4. [button] 10002	CLICK
5. [button] Search	CLICK
6. [switch] Show BBB Accredited only	CLICK
7. [svg]	CLICK
8. [button] Sort By	CLICK
9. [link] Fast Lane 24 Hour Auto Repair	CLICK
10. [link] Read Reviews	CLICK

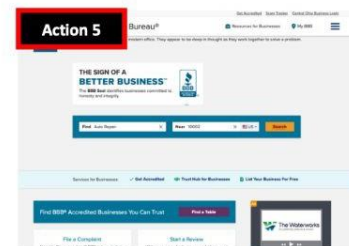
Webpage Snapshots:



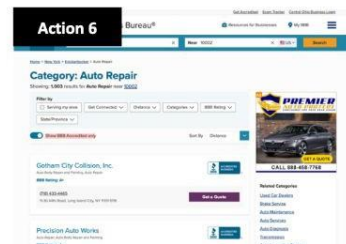
`<input name="find_text" type="search">`



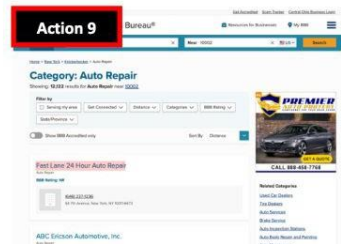
`<em>Auto Repair</em>`



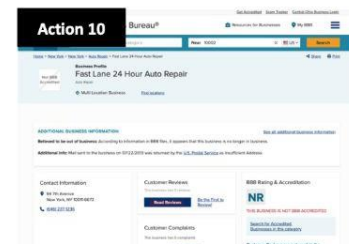
`<button>Search</button>`



`<button>Show BBB Accredited only</button>`

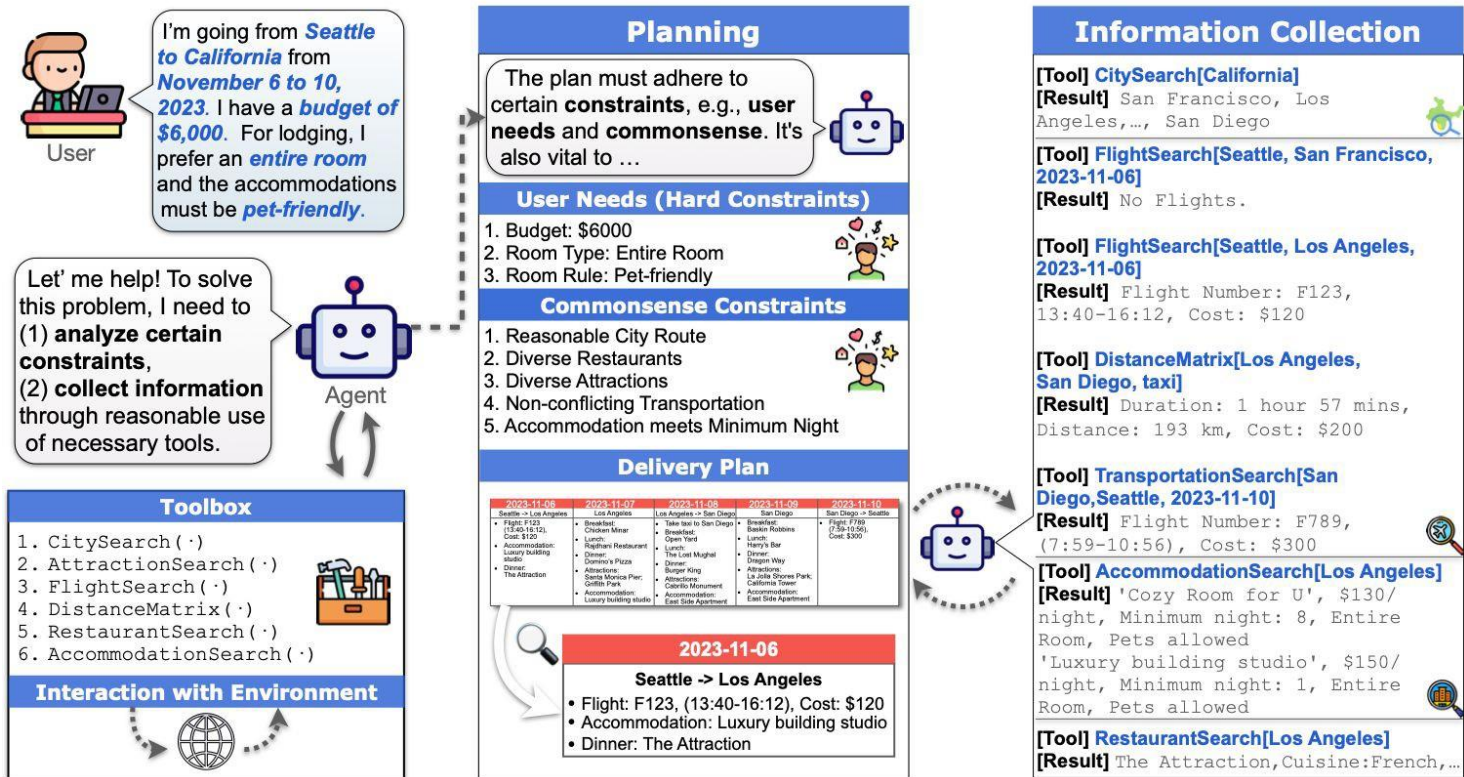


`<span>Fast Lane 24 Hour Auto Repair</span>`



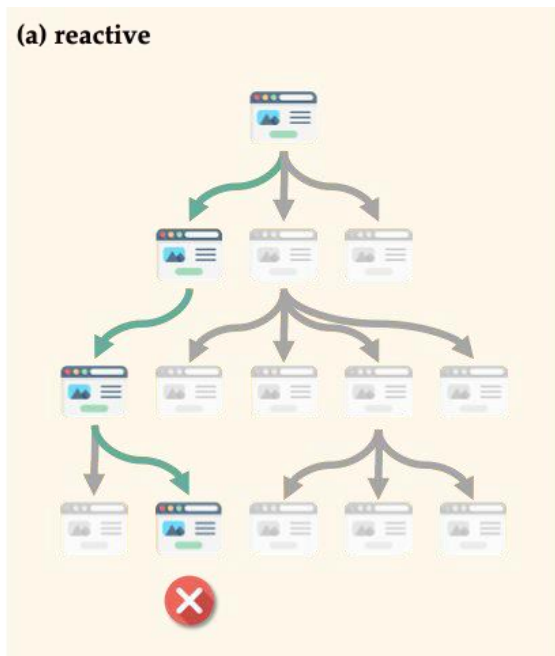
`<a href="link:XXX">Read Reviews</a>`

Travel planning with several constraints

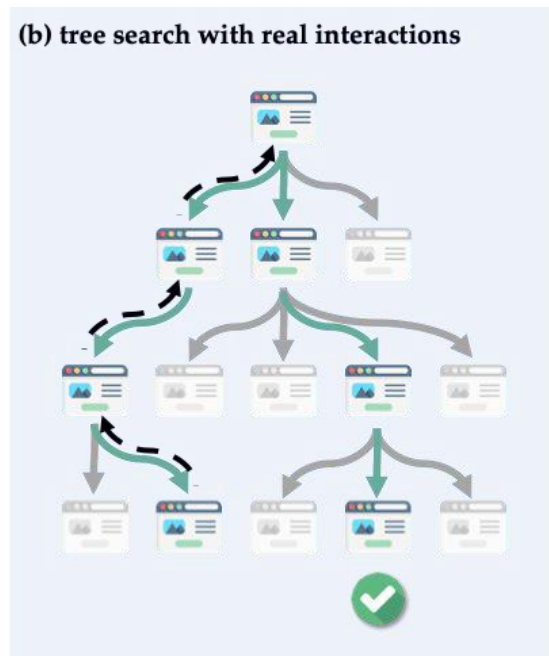


Reactive action selection and tree search

(a) reactive



(b) tree search with real interactions

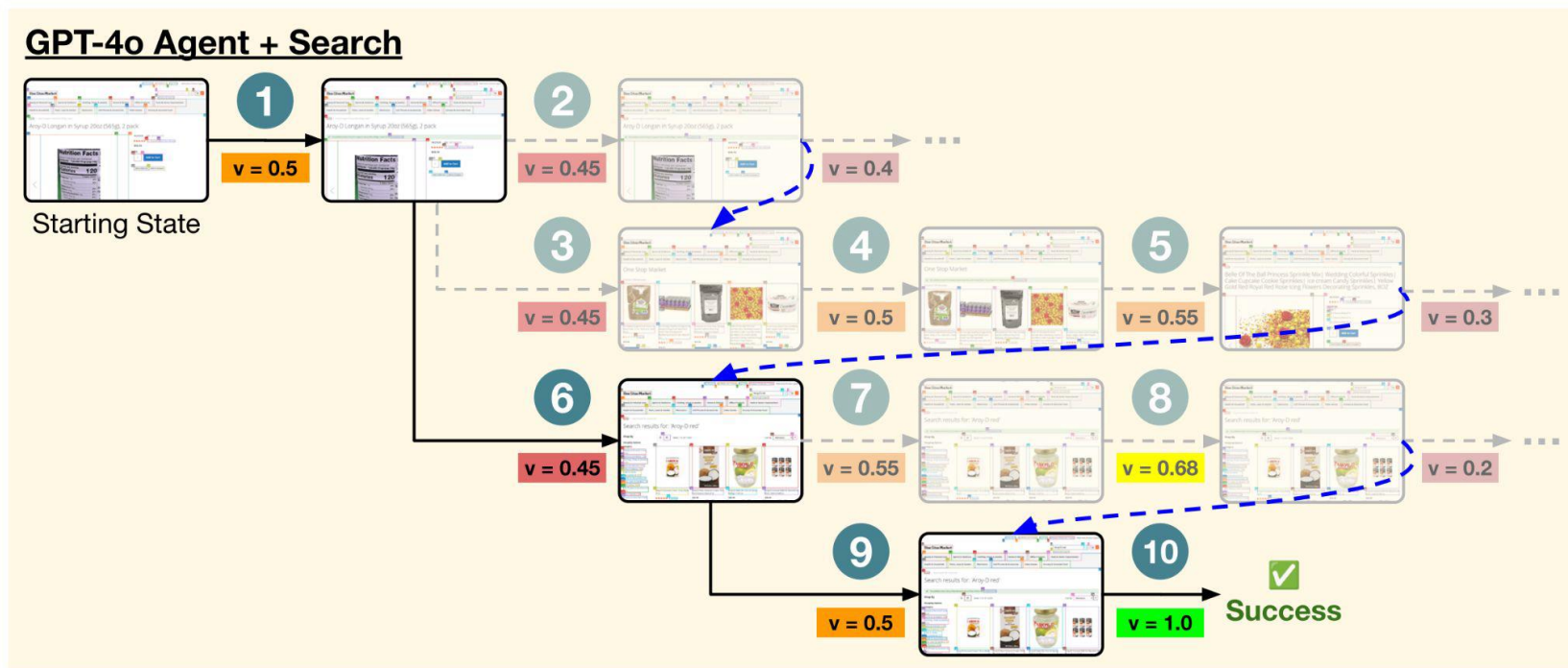


Fast and easy to implement



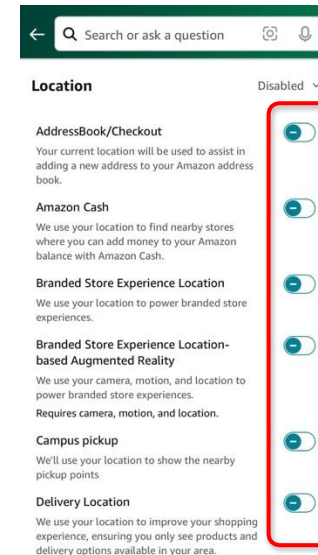
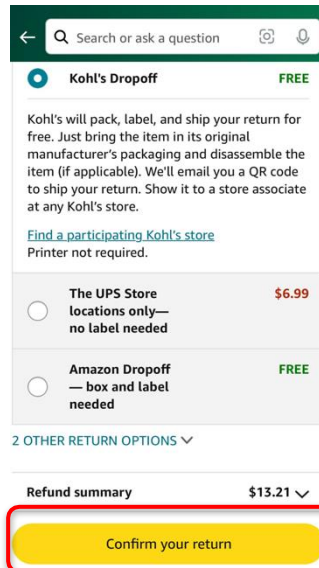
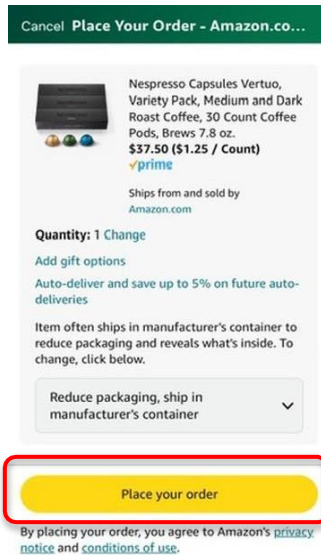
Limited lookahead

Tree search with real interactions



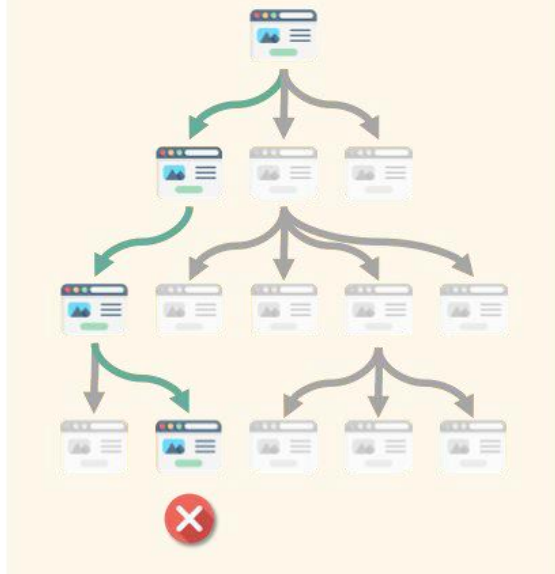
Some actions cannot be undone

A purchase or return can trigger a real transaction.
Exploration may expose private data or create safety risks.
Trying many actions also takes time and money.



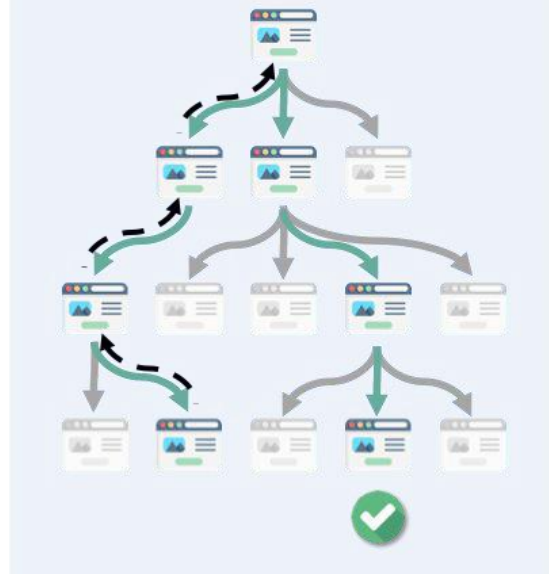
Planning with a model of the environment

(a) reactive



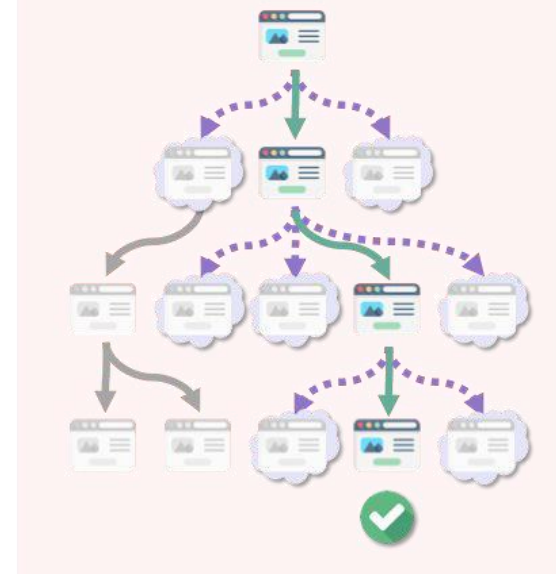
-  Fast and simple
-  Limited lookahead

(b) tree search with real interactions



-  Explores alternatives
-  Needs safe backtracking

(c) model-based planning



-  Simulates alternatives
-  Needs a reliable model

A world model predicts state transitions

A computational model of how actions change the environment

$$\hat{T}: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$$

Action

a_t

Current state

s_t

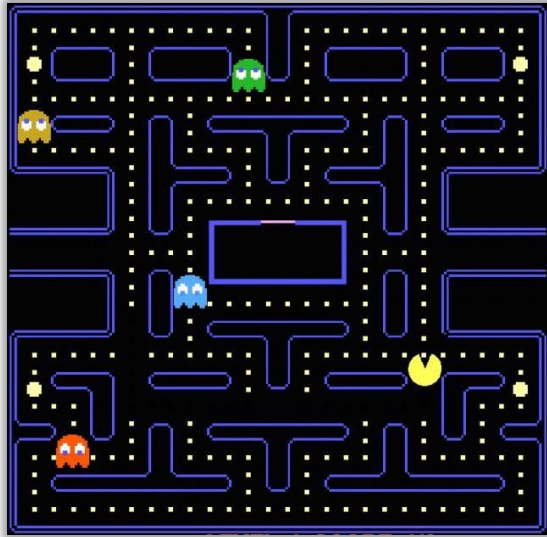
Next state

s_{t+1}

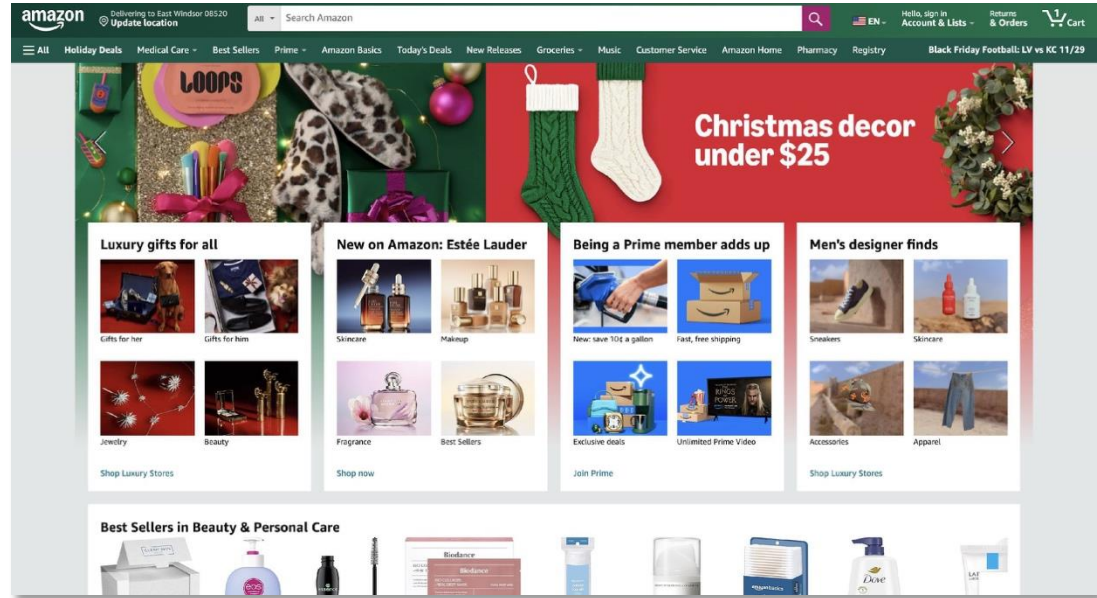
This deterministic approximation predicts one next state.

A stochastic model predicts a distribution over next states.

Modeling the open web

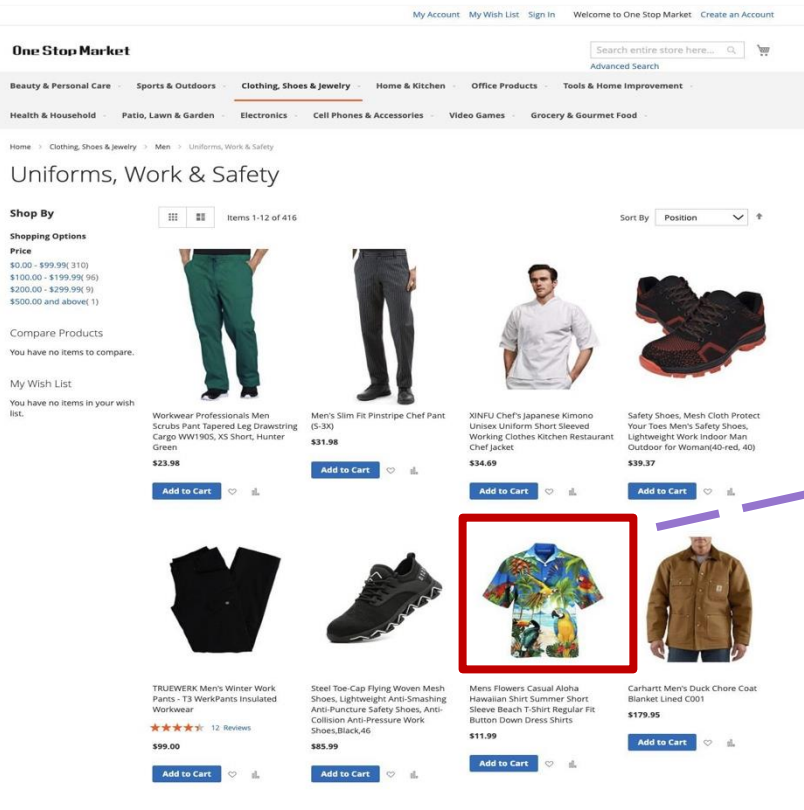


V.S



Websites differ in their actions and page structure.

Predicting a page change



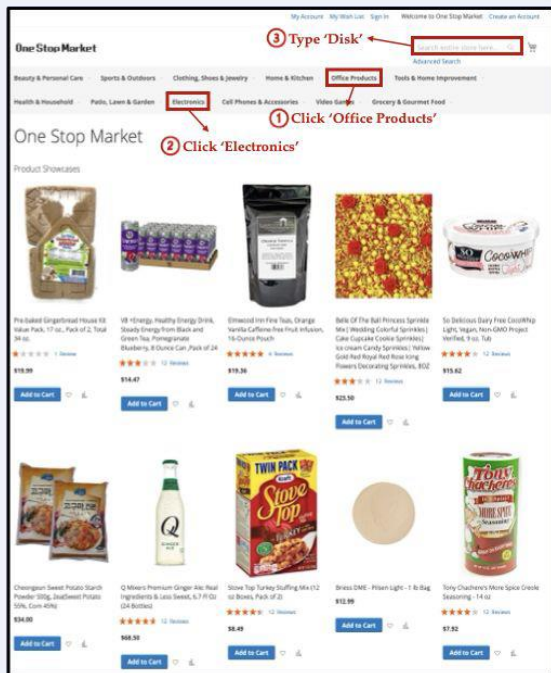
Example prediction

Clicking the shirt opens its product page. It may show specifications, reviews, larger images, size options, and an Add to Cart button.

The product details replace the grid of search results.

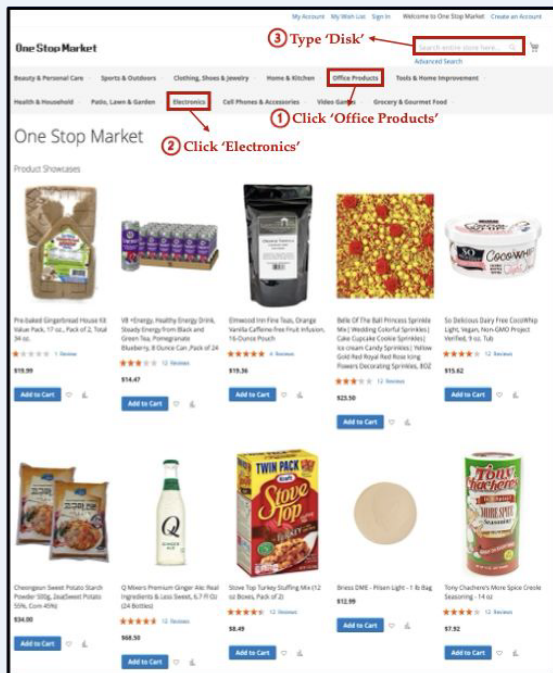
WebDREAMER: candidate actions

Please navigate to the 'Data Storage' category and purchase the least expensive disk with 512GB of storage.



WebDREAMER: the first predicted step

Please navigate to the 'Data Storage' category and purchase the least expensive disk with 512GB of storage.

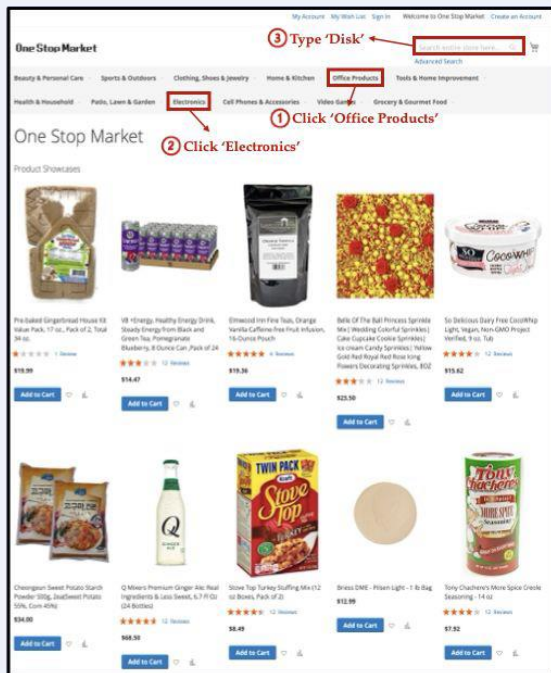


Stage I: Simulation

1 The 'Office Products' category will display three sub-categories: 'Office Electronics', 'Office & School Supplies', and 'Office Furniture & Lighting'.

WebDremer: two-step lookahead

Please navigate to the 'Data Storage' category and purchase the least expensive disk with 512GB of storage.

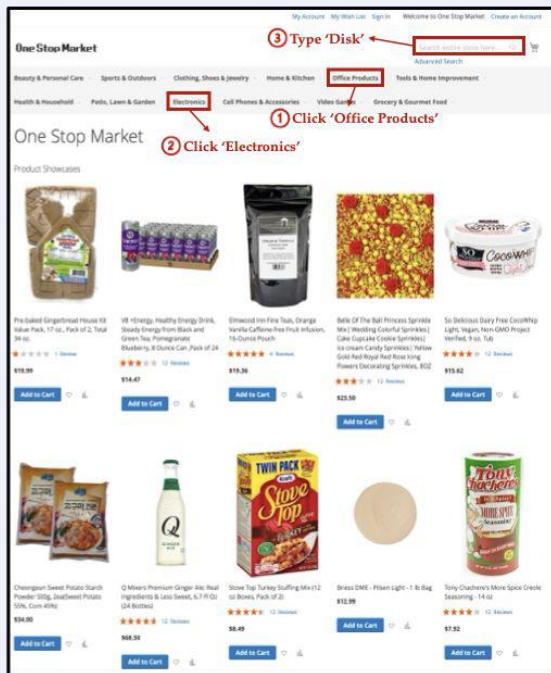


Stage I: Simulation

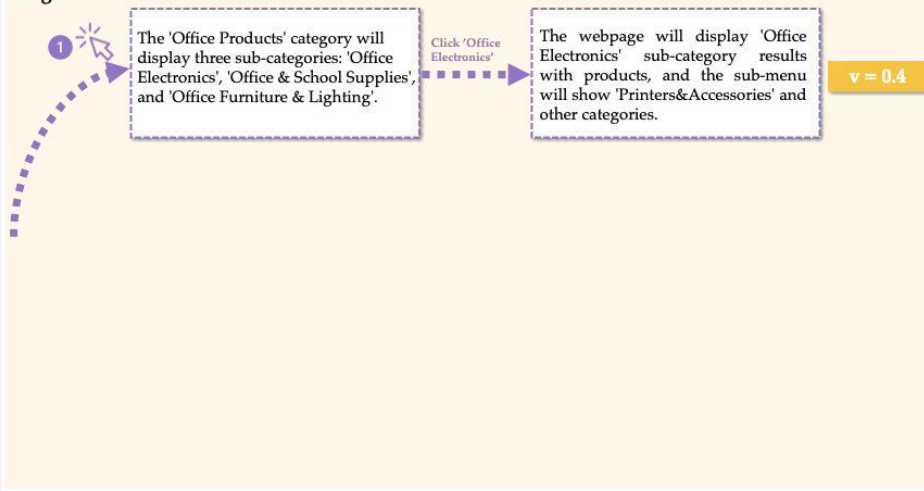


WebDreamer: scoring a trajectory

Please navigate to the 'Data Storage' category and purchase the least expensive disk with 512GB of storage.

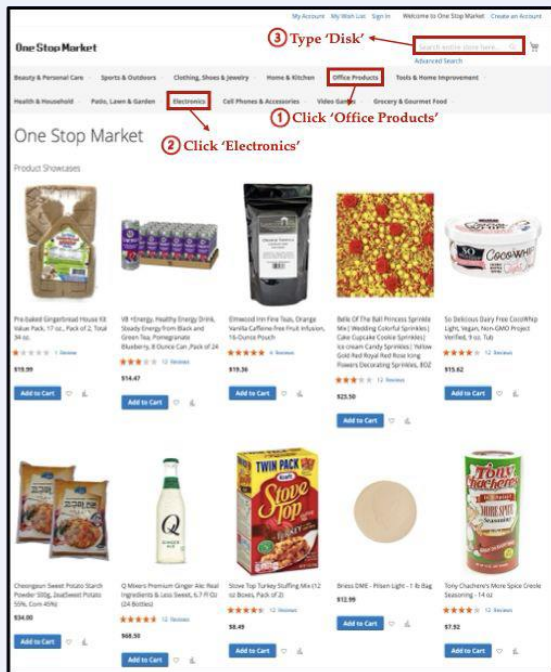


Stage I: Simulation

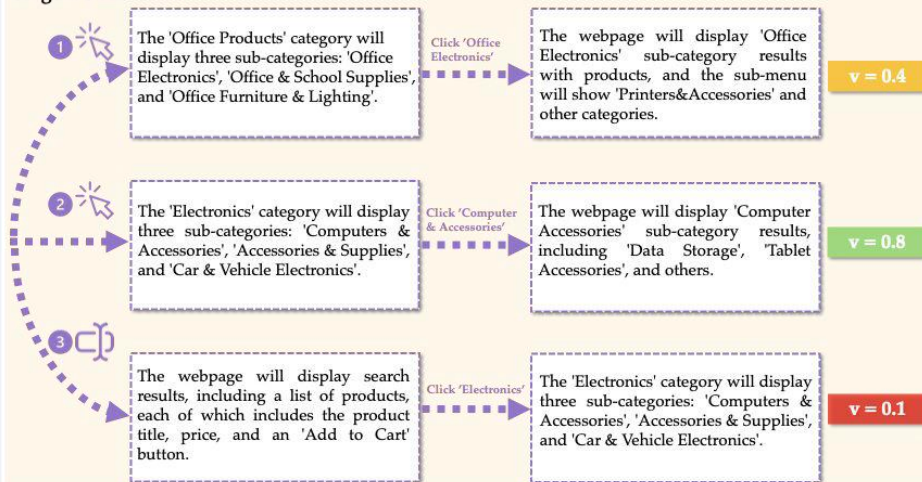


WebDreamer: choosing an action

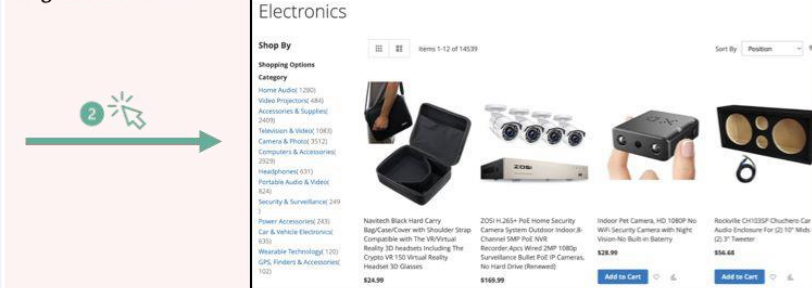
Please navigate to the 'Data Storage' category and purchase the least expensive disk with 512GB of storage.



Stage I: Simulation



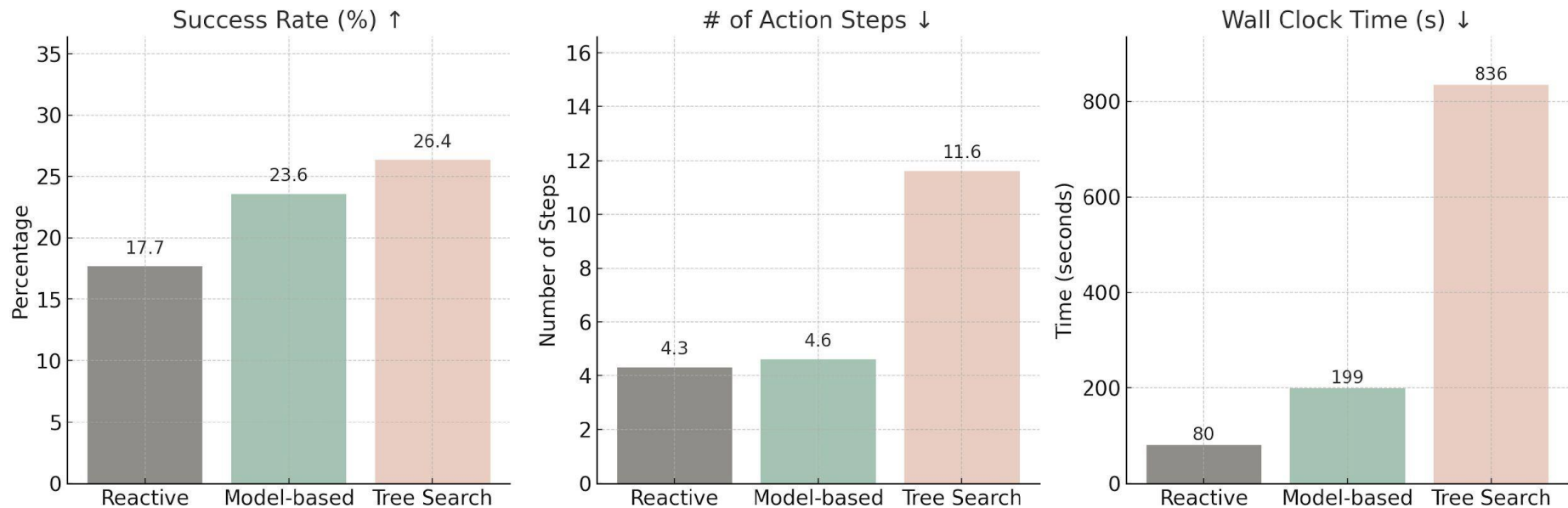
Stage II: Execution



WebDremer results on VisualWebArena

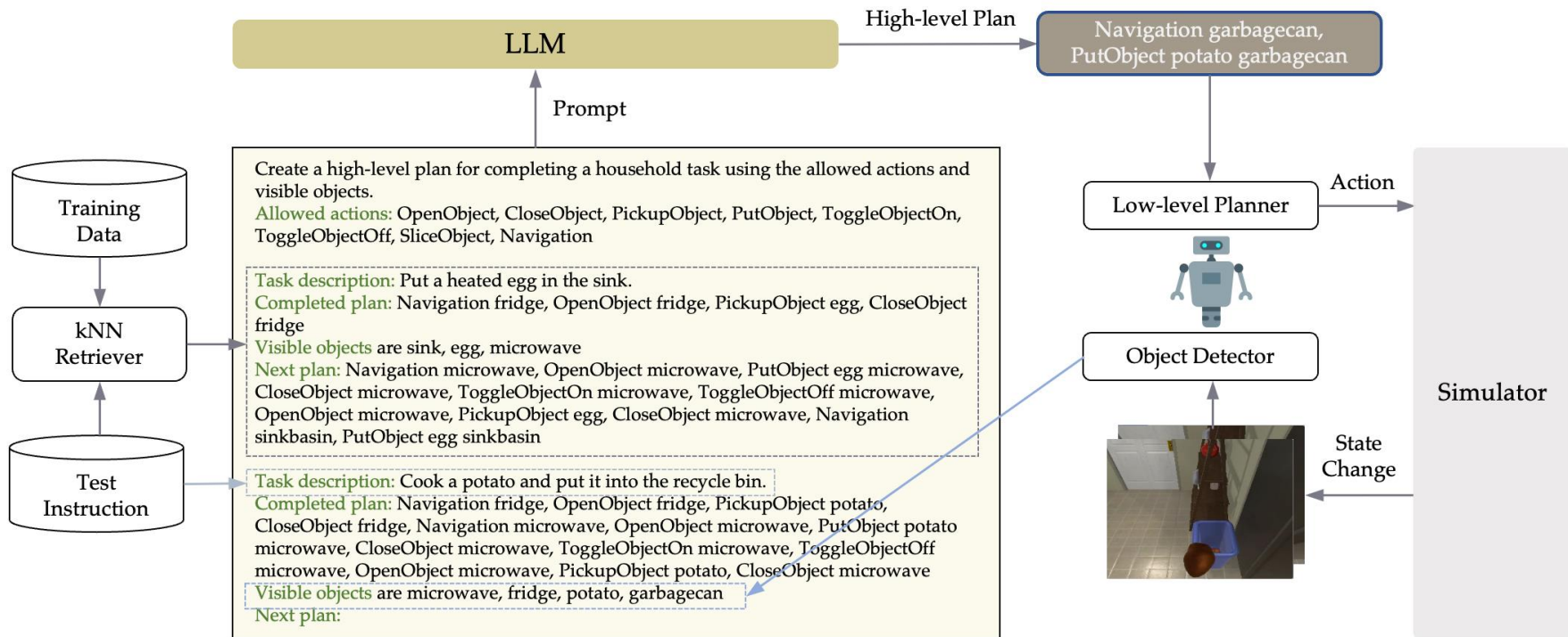
Model-based planning improves success over the reactive baseline in this study.

Tree search succeeds more often, with more actions and longer runtime.

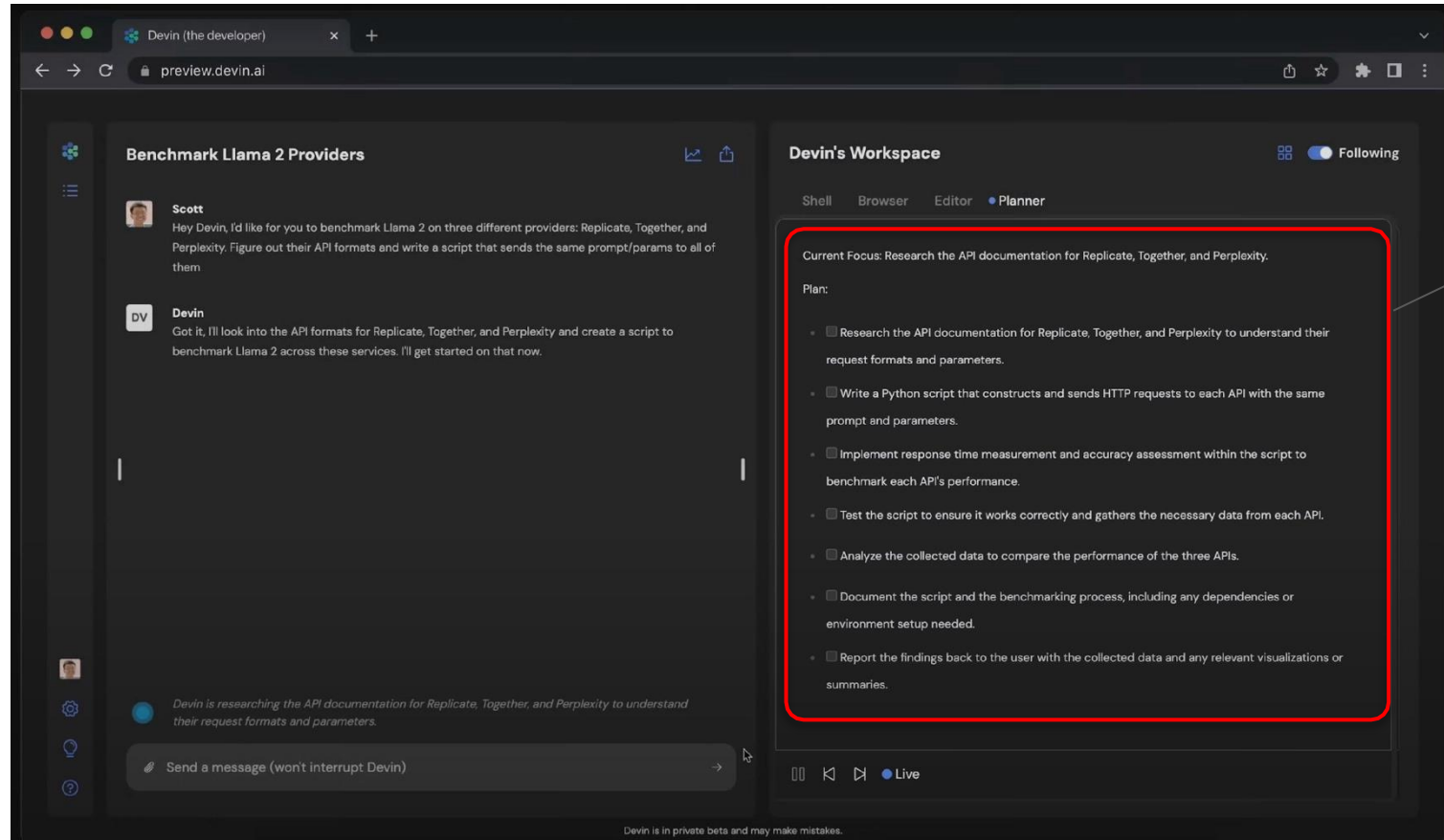


Reactive and tree-search baselines: Koh et al. (2024)

LLM-Planner: grounded replanning



A coding agent's initial plan



The screenshot shows the Devin AI developer interface. On the left, a chat window titled "Benchmark Llama 2 Providers" shows a conversation between Scott and Devin. Scott asks Devin to benchmark Llama 2 on three providers: Replicate, Together, and Perplexity. Devin responds that it will look into the API formats and create a script. Below the chat, a status bar indicates that Devin is researching the API documentation for the three providers.

On the right, the "Devin's Workspace" is shown with a "Planner" tab selected. The planner displays a "Current Focus" and a "Plan" with eight steps, each with a checkbox. A red box highlights the entire planner section, and an arrow points from the text "Initial plan" to this box.

Benchmark Llama 2 Providers

Scott
Hey Devin, I'd like for you to benchmark Llama 2 on three different providers: Replicate, Together, and Perplexity. Figure out their API formats and write a script that sends the same prompt/params to all of them

Devin
Got it, I'll look into the API formats for Replicate, Together, and Perplexity and create a script to benchmark Llama 2 across these services. I'll get started on that now.

Devin is researching the API documentation for Replicate, Together, and Perplexity to understand their request formats and parameters.

Send a message (won't interrupt Devin)

Devin's Workspace

Shell Browser Editor **Planner**

Current Focus: Research the API documentation for Replicate, Together, and Perplexity.

Plan:

- ☐ Research the API documentation for Replicate, Together, and Perplexity to understand their request formats and parameters.
- ☐ Write a Python script that constructs and sends HTTP requests to each API with the same prompt and parameters.
- ☐ Implement response time measurement and accuracy assessment within the script to benchmark each API's performance.
- ☐ Test the script to ensure it works correctly and gathers the necessary data from each API.
- ☐ Analyze the collected data to compare the performance of the three APIs.
- ☐ Document the script and the benchmarking process, including any dependencies or environment setup needed.
- ☐ Report the findings back to the user with the collected data and any relevant visualizations or summaries.

Devin is in private beta and may make mistakes.

Initial
plan

An unexpected API response

The screenshot displays the Devin AI developer interface. On the left, a chat window shows a conversation between Scott and Devin. Scott asks if the correct model names are being used, and Devin responds that they have updated the Replicate model ID and will check the model names for Together and Perplexity. Below the chat, a URL bar shows <https://docs.together.ai/reference/chat-completions>. The main workspace area contains a code editor. The top editor, titled 'default', shows a traceback error: 'Command 'python' not found, did you mean: command 'python3' from deb python3', 'command 'python' from deb python-is-python3'. The error occurs in the file `/home/ubuntu/benchmark.py` at line 39, where the code attempts to access `response.json()[0]['choices'][0]['text'].split()`, resulting in a `KeyError: 'text'`. A red box highlights this error message, and an arrow points to it with the text 'Missing "text" field'. The bottom editor, titled 'benchmark.py', shows the Python code for the benchmark script, which imports `os`, `time`, and `requests`, and defines API endpoints for Replicate, Together, and Perplexity.

Devin's Workspace

Scott
can you make sure you use the right model names? i want them to all be llama 2 70b.

Devin
Hey Scott, absolutely, I'll ensure that all the model names used in the benchmarking script are for Llama 2 70b. I've already updated the Replicate model ID with the one you provided. I'll double-check the model names for Together and Perplexity to make sure they're correct and update them if necessary. I'm currently troubleshooting an issue with the script and will proceed with the testing as soon as it's resolved. Thanks for the reminder!

Devin is currently debugging the script to handle API response errors and ensure accurate parsing of the output.

default

```
Command 'python' not found, did you mean:
command 'python3' from deb python3
command 'python' from deb python-is-python3
ubuntu@ip-172-31-27-196:~$ python3 benchmark.py
Error: 422
Traceback (most recent call last):
  File "/home/ubuntu/benchmark.py", line 65, in <module>
    together_tps = benchmark_api(TOGETHER_ENDPOINT, TOGETHER_HEADERS, together_d
ata)
  File "/home/ubuntu/benchmark.py", line 39, in benchmark_api
    tokens = len(response.json()[0]['choices'][0]['text'].split())
KeyError: 'text'
ubuntu@ip-172-31-27-196:~$
```

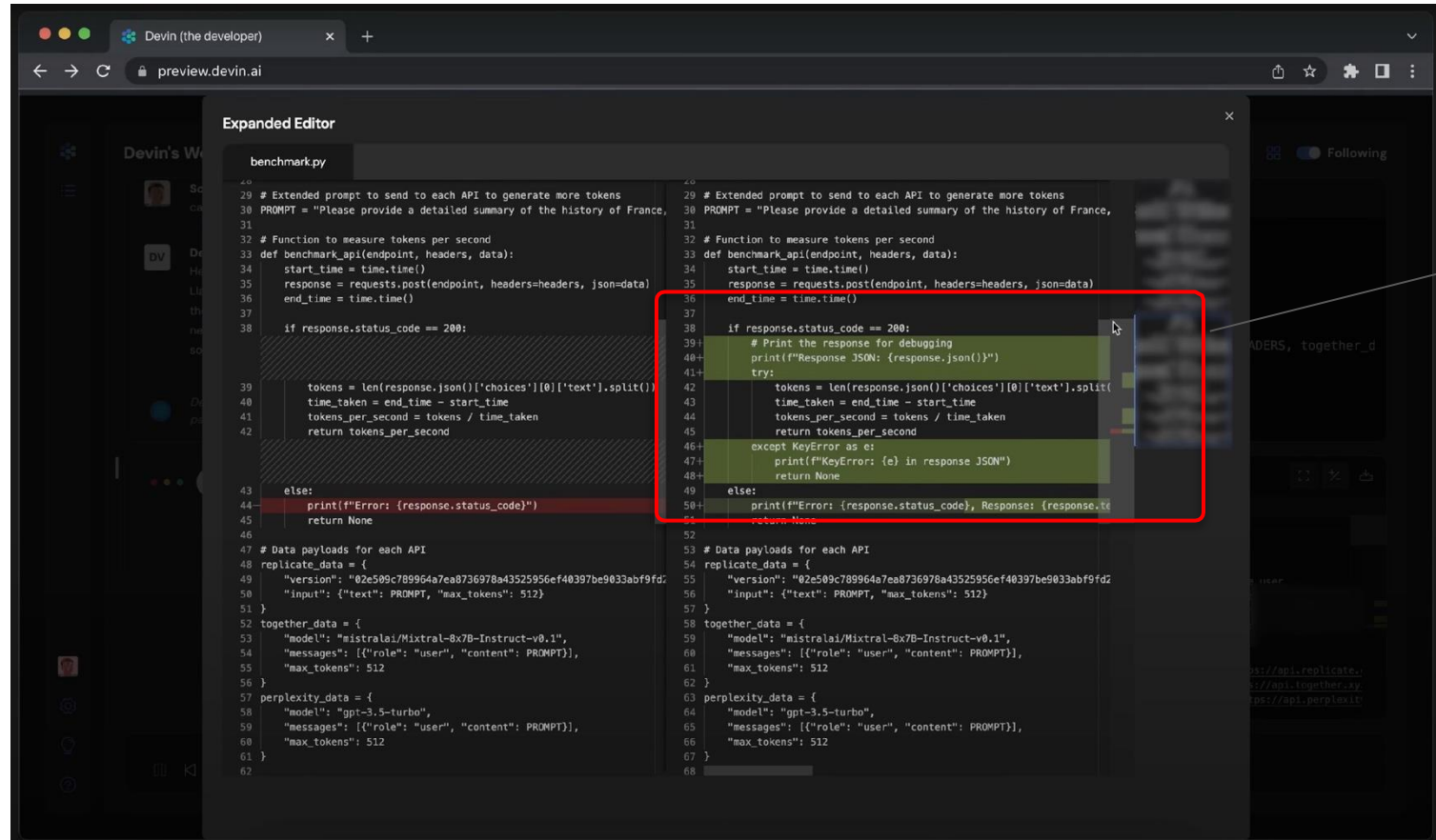
Expand editor

benchmark.py

```
home > ubuntu > benchmark.py
1 import os
2 import time
3 import requests
4
5 # API keys provided by the user
6 TOGETHER_API_KEY = "c2"
7 REPLICATE_API_KEY = "i"
8 PERPLEXITY_API_KEY = "i"
9
10 # API endpoints
11 REPLICATE_ENDPOINT = "https://api.replicate.com/v1"
12 TOGETHER_ENDPOINT = "https://api.together.xyz"
13 PERPLEXITY_ENDPOINT = "https://api.perplexity.ai"
14
```

Missing
'text'
field

The agent inspects the response

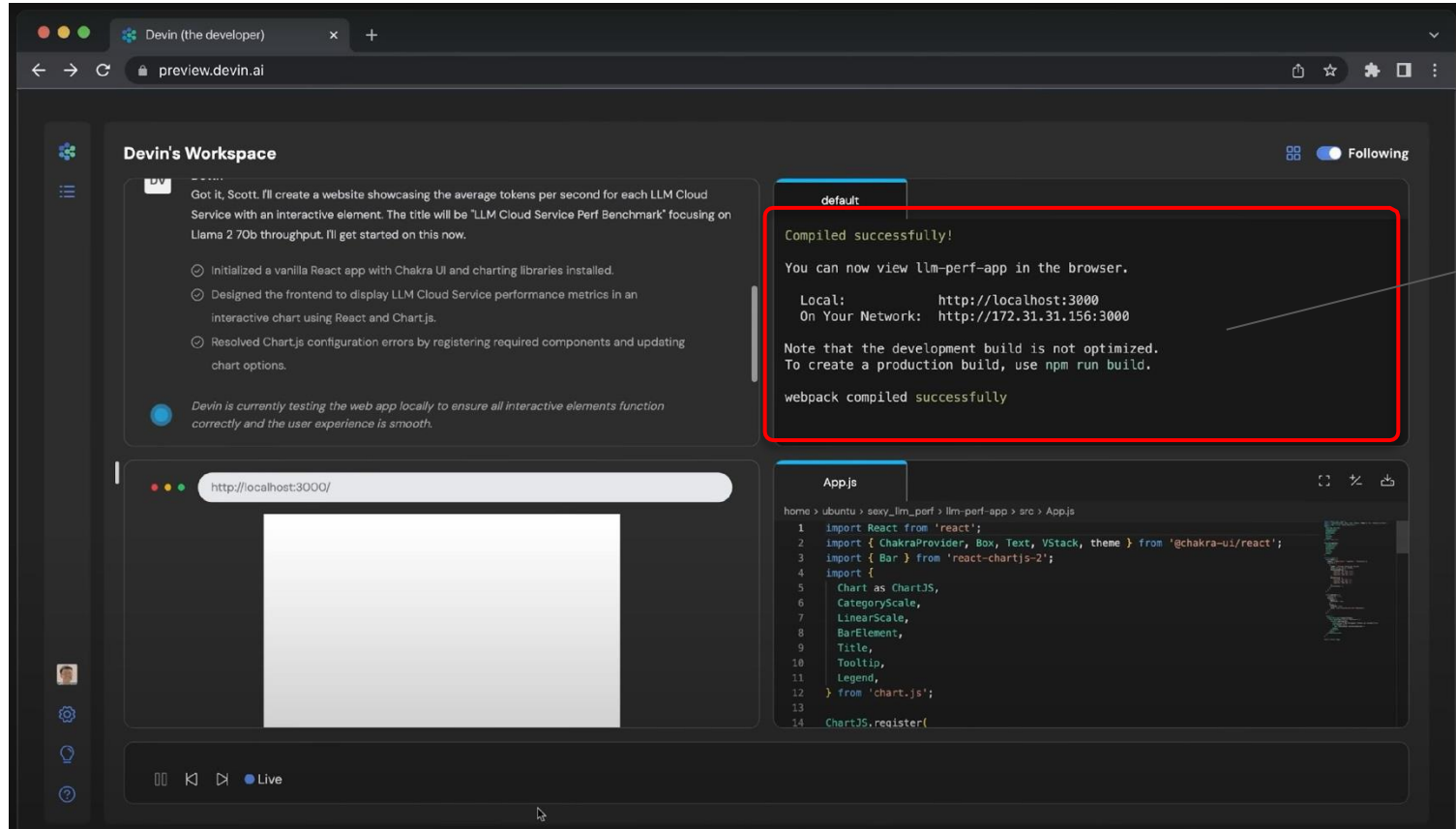


The screenshot shows the Devin AI developer interface. The main window is the 'Expanded Editor' displaying a Python file named 'benchmark.py'. The code is split into two panes, showing the same code from different perspectives. A red rectangular box highlights a specific section of the code in the right pane, which is also visible in the left pane. This section contains a conditional check for the response status code and a try-except block for handling JSON parsing errors. An arrow points from the text 'Inspect JSON' to this highlighted section.

```
40 # Extended prompt to send to each API to generate more tokens
38 PROMPT = "Please provide a detailed summary of the history of France,
31
32 # Function to measure tokens per second
33 def benchmark_api(endpoint, headers, data):
34     start_time = time.time()
35     response = requests.post(endpoint, headers=headers, json=data)
36     end_time = time.time()
37
38     if response.status_code == 200:
39
40         tokens = len(response.json()['choices'][0]['text'].split())
41         time_taken = end_time - start_time
42         tokens_per_second = tokens / time_taken
43         return tokens_per_second
44
45     else:
46         print(f"Error: {response.status_code}")
47         return None
48
49 # Data payloads for each API
50 replicate_data = {
51     "version": "02e509c789964a7ea8736978a43525956ef48397be9033abf9fdz",
52     "input": {"text": PROMPT, "max_tokens": 512}
53 }
54
55 together_data = {
56     "model": "mistralai/Mistral-8x7B-Instruct-v0.1",
57     "messages": [{"role": "user", "content": PROMPT}],
58     "max_tokens": 512
59 }
60
61 perplexity_data = {
62     "model": "gpt-3.5-turbo",
63     "messages": [{"role": "user", "content": PROMPT}],
64     "max_tokens": 512
65 }
66
67
68
```

Inspect
JSON

A later step in the demo



The app
compiles

Planning: what to keep in mind

Goals can be ambiguous. Actions may be open-ended, and success may be hard to check.

A world model supports lookahead. Its errors can also lead the planner astray.

A plan needs feedback. New observations can change which subgoal or action makes sense.

The best strategy depends on the model and task. Planning quality remains an open research problem.